



Program Studi Bioinformatika
FMIPA IPB University

Modul Praktikum Sistem Basis Data Bioinformatika

Pertemuan 5 SQL Lanjut dan Query Biologis

Prof. Dr. Eng. Wisnu Ananta Kusuma, S.T., M.T.
Dr. Luthfan Irfana, S.Si., M.Si.
Lailan Sahrina Hasibuan, S.Kom., M.Kom.
Gilland Fausta Putra Achyar, S.Mat., M.Kom.
Said Thaufik Rizaldi, S.Kom., M.Kom.
Hakim Ilyas Azhar
Tsabitha Naylasafa Aurora

Daftar Isi

1	Pendahuluan	1
2	Memahami Data dan Desain Relasional	1
2.1	Konteks Dataset BioSQLite	1
2.2	Desain Relasional BioSQLite	2
2.3	Prapraktikum	3
3	Menyiapkan Database Tujuan	3
3.1	Membuat Database Praktikum	3
3.2	Menyiapkan Schema Kosong untuk Restore	4
4	Memulihkan Struktur dan Data melalui pgAdmin	4
4.1	Memilih Arsip Backup	5
4.2	Menjalankan dan Memeriksa Restore	5
5	Memeriksa Kelengkapan Hasil Restore.	6
5.1	Memeriksa Jumlah Baris	6
5.2	Memeriksa Primary Key dan Foreign Key	7
6	Menggunakan Query Tool dan Membaca Query.	8
7	INNER JOIN untuk Data yang Tersedia Bersama	9
7.1	Menggabungkan Data RNA-seq dan Mutagenesis	9
7.2	Memeriksa Jumlah Pasangan dan Interpretasi	10
8	Menghubungkan Identifier dengan Nama Pathway	11
8.1	Menelusuri Hubungan Tiga Tabel	11
8.2	Memeriksa Jumlah Identifier yang Berbeda	12
9	LEFT JOIN dan RIGHT JOIN untuk Mempertahankan Katalog	12
9.1	LEFT JOIN dan Identifier tanpa Pasangan	12
9.2	Menyatakan Tujuan yang Sama dengan RIGHT JOIN	13
10	Menempatkan Filter pada ON atau WHERE	13
11	FULL OUTER JOIN untuk Membandingkan Cakupan Sumber	14
12	CROSS JOIN dan Self Join.	16
13	Merangkum Hasil dengan GROUP BY dan HAVING.	17
14	Subquery untuk Pertanyaan Bertingkat	19
15	Memeriksa Pertambahan Baris pada JOIN Anotasi	20
15.1	Menggabungkan Pathway dengan Anotasi GO	20
16	Mengatasi Kendala yang Sering Muncul.	22
17	Latihan Individual dan Refleksi	23
17.1	Latihan Individual	23
17.2	Refleksi Akhir	23
18	Lembar Kerja Praktikum	23
18.1	Berkas yang Dikumpulkan dan Skrip SQL	23
18.2	Struktur dan Ketentuan Laporan Praktikum	24

Tujuan Praktikum

1. Memilih dan menggunakan INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN, dan *self join* sesuai kebutuhan penggabungan data.
2. Menggunakan WHERE, ORDER BY, fungsi agregasi, GROUP BY, HAVING, dan *subquery* untuk menjawab pertanyaan pada data biologis.
3. Memeriksa kardinalitas, nilai NULL, dan penambahan baris, kemudian menginterpretasikan hasil *query* sesuai cakupan dataset.

1 Pendahuluan

Pada Pertemuan 4, mahasiswa telah mempelajari perintah SQL dasar untuk membuat struktur serta menambahkan, membaca, mengubah, dan menghapus data. Pertemuan 5 melanjutkan kemampuan tersebut dengan menggunakan *query* untuk menghubungkan dan merangkum informasi dari beberapa tabel.

Praktikum menggunakan basis data BioSQLite yang memuat data genom, RNA-seq, mutagenesis, anotasi *Gene Ontology*, dan *pathway Plasmodium falciparum*. Basis data dipulihkan dari berkas cadangan PostgreSQL yang disediakan sehingga mahasiswa dapat langsung bekerja dengan dataset yang saling berhubungan.

Setiap pertanyaan diterjemahkan menjadi pilihan tabel, kolom pencocokan, kondisi penyaringan, jenis penggabungan, dan bentuk hasil yang diperlukan. Mahasiswa kemudian membandingkan hasil *query* dengan prediksi serta menafsirkan hasil sesuai cakupan dataset. *Screenshot* digunakan sebagai bukti pelaksanaan, sedangkan penjelasan tertulis menunjukkan pemahaman terhadap hasil.

2 Memahami Data dan Desain Relasional

2.1 Konteks Dataset BioSQLite

Praktikum menggunakan satu *snapshot* data *Plasmodium falciparum* yang mencakup fitur genom, *raw count* RNA-seq tiga ulangan WT dan tiga ulangan KO, skor mutagenesis, anotasi *Gene Ontology* (GO), serta pasangan *identifier-pathway*. “Dataset lengkap” berarti seluruh baris dari berkas sumber praktikum telah dimuat ke dalam tujuh tabel, bukan seluruh pengetahuan biologis mengenai organisme tersebut. Tabel hasil analisis lanjutan pada repositori tidak termasuk dalam paket praktikum.

Dataset ini disusun dalam tujuh tabel. Arti satu baris dan jumlah baris pada setiap tabel dirangkum pada Tabel 1.

Tabel 1. Ringkasan tabel dalam database BioSQLite praktikum.

Tabel	Satu baris mewakili	Jumlah
gene	Satu <i>identifier</i> dari gabungan sumber	5.750
gene_feature	Satu fitur pada sekuens referensi	49.712
reads	<i>Count</i> enam sampel untuk satu <i>identifier</i>	5.670
mis_essential	Skor mutagenesis untuk satu <i>identifier</i>	5.399
go_annotation	Satu rekaman anotasi GO	10.655
pathways	Satu entri <i>pathway</i>	106

Tabel 1. Ringkasan tabel dalam database BioSQLite praktikum (lanjutan).

Tabel	Satu baris mewakili	Jumlah
gene_pathway	Satu pasangan <i>identifier-pathway</i>	2.284

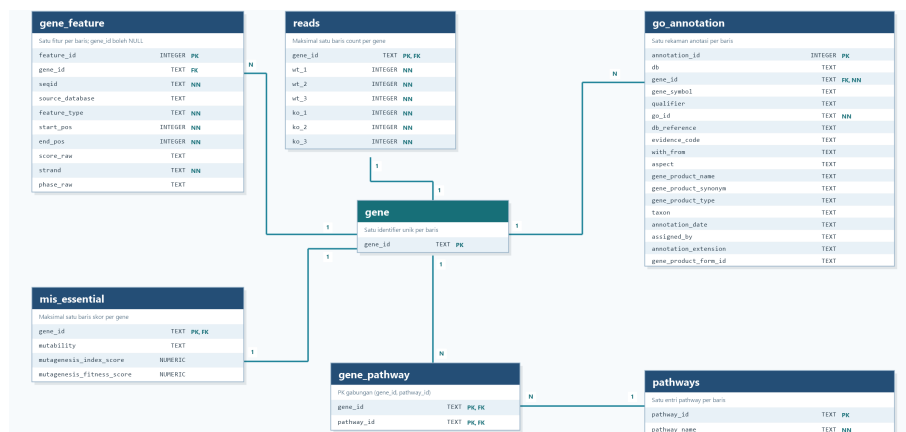
Jumlah baris berbeda karena setiap tabel memiliki cakupan dan satuan pencatatan yang berbeda. Satu *identifier* dapat mempunyai banyak fitur genom, anotasi GO, atau hubungan dengan beberapa *pathway*.

WT (*wild type*) merupakan kelompok acuan, sedangkan KO (*knockout*) merupakan kelompok dengan gen sasaran yang dinonaktifkan. *Raw count* adalah jumlah *read* yang dialokasikan ke suatu fitur. GO memuat anotasi fungsi, proses biologis, dan lokasi seluler, sedangkan *pathway* menggambarkan keterlibatan dalam lintasan biologis.

2.2 Desain Relasional BioSQLite

Satu *identifier* dapat berkaitan dengan beberapa fitur genom sehingga **gene_id** pada **gene_feature** tidak unik. Tabel **gene** menyediakan **gene_id** yang unik sebagai *primary key*. Tabel **reads** dan **mis_essential** masing-masing memiliki paling banyak satu baris per *identifier*, sedangkan **gene_feature** dan **go_annotation** dapat memiliki banyak baris dan menggunakan ID internal sebagai *primary key*. Sebanyak 9.348 fitur tanpa **gene_id** tetap disimpan pada **gene_feature** dengan nilai NULL. Tabel **gene_pathway** menjembatani hubungan banyak-ke-banyak antara **gene** dan **pathways** menggunakan *primary key* gabungan (**gene_id**, **pathway_id**). Paket ini memuat 7 *primary key* dan 6 *foreign key*.

Hubungan ketujuh tabel ditunjukkan pada Gambar 1. Tabel **gene** menjadi induk *identifier* bagi lima tabel, sedangkan **pathways** menjadi induk kedua bagi **gene_pathway**. Panah mengarah dari tabel anak ke tabel induk yang dirujuk. PK menandai *primary key*, FK menandai *foreign key*, dan NN menandai kolom NOT NULL.



Gambar 1. Relasi database BioSQLite.

Batas interpretasi perlu diperhatikan saat membaca hasil query. Sebanyak 5.750 *identifier* pada tabel **gene** belum tentu mewakili 5.750 gen biologis unik. Nama dari berbagai sumber, termasuk akhiran seperti .1, dipertahankan karena kesetaraannya belum seluruhnya divalidasi. *Identifier* yang tidak memperoleh pasangan dalam JOIN dapat menunjukkan perbedaan cakupan atau penamaan antarsumber. Hasil *query* juga tidak langsung membuktikan perubahan ekspresi, esensialitas, atau aktivitas suatu *pathway*.

2.3 Prapraktikum

Setelah mempelajari konteks dataset dan desain relasional BioSQLite, jawab pertanyaan berikut secara individual sebelum mengikuti kegiatan praktikum.

Tugas Prapraktikum

1. Jelaskan perbedaan antara satu gen dan satu fitur pada model data praktikum.
2. Jelaskan alasan tabel `gene_pathway` memerlukan dua kolom penyusun *primary key*.
3. Jelaskan dugaan makna nilai `NULL` yang muncul pada hasil penggabungan tabel.
4. Tuliskan dua penjelasan yang mungkin ketika *identifier* dari suatu sumber tidak ditemukan pada sumber lain.

Jawaban ditulis tangan secara singkat menggunakan kalimat sendiri pada selembar kertas utuh, kemudian dibawa untuk diperiksa dan didiskusikan pada awal praktikum. Tugas prapraktikum ini tidak dicantumkan dalam LKP.

3 Menyiapkan Database Tujuan

Gunakan PostgreSQL 18, pgAdmin 4, dan berkas `BioSQLite_Mg05_v1.backup` yang telah disediakan. Simpan berkas `.backup` pada folder lokal yang mudah ditemukan. Berkas tersebut merupakan arsip PostgreSQL format Custom dan tidak dibuka sebagai skrip SQL.

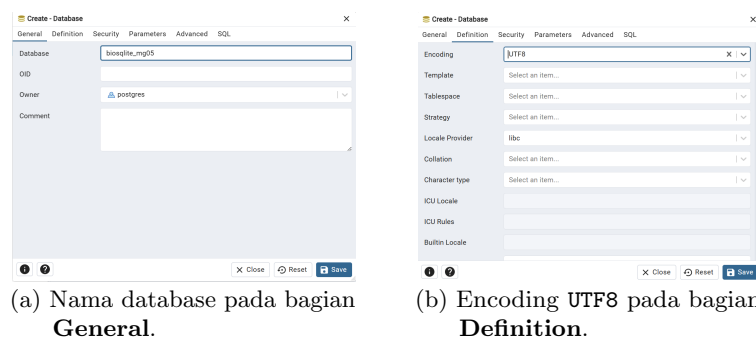
3.1 Membuat Database Praktikum

Database praktikum diberi nama `biosqlite_mg05`.

Buat database tersebut melalui langkah berikut.

1. Buka pgAdmin, kemudian buka server PostgreSQL yang telah disiapkan. Masukkan kata sandi yang dibuat saat instalasi jika diminta.
2. Pada *Object Explorer*, klik kanan **Databases**, lalu pilih **Create** → **Database**.
3. Isi **Database** dengan `biosqlite_mg05`. Gunakan **Owner** sesuai akun yang tersedia; pada instalasi pribadi biasanya `postgres`.
4. Pada bagian **Definition**, pastikan **Encoding** bernilai `UTF8`. Biarkan pengaturan lain mengikuti konfigurasi PostgreSQL, kemudian klik **Save**.

Pengisian nama database dan pemilihan encoding ditunjukkan pada Gambar 2.

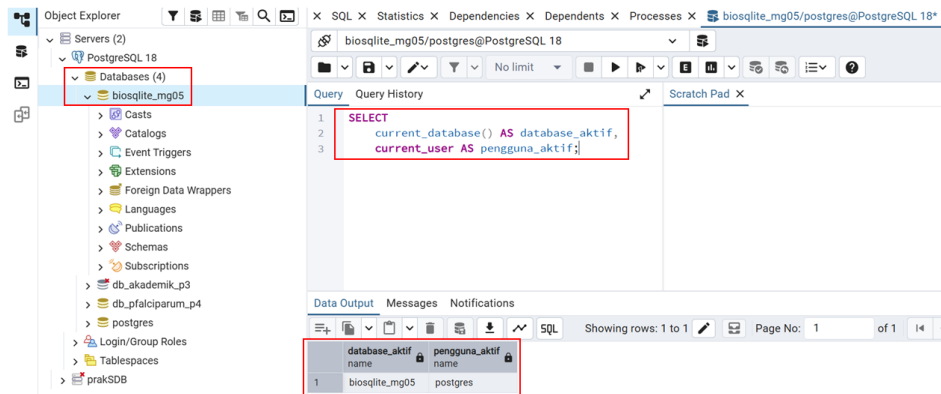


Gambar 2. Pembuatan database praktikum.

Klik kanan biosqlite_mg05, lalu pilih **Query Tool**. Sebelum menjalankan query, pastikan nama database pada tab atau bilah koneksi adalah biosqlite_mg05.

```
SELECT
    current_database() AS database_aktif,
    current_user AS pengguna_aktif;
```

Nilai database_aktif harus menunjukkan biosqlite_mg05. Jika berbeda, buka kembali Query Tool dari database yang benar. Hasil pemeriksaan database aktif ditunjukkan pada Gambar 3.



Gambar 3. Verifikasi database dan pengguna aktif.

3.2 Menyiapkan Schema Kosong untuk Restore

Arsip praktikum memuat perintah pembuatan schema public, sedangkan database baru telah mempunyai schema public bawaan. Pastikan database praktikum masih baru dan belum mempunyai tabel buatan pengguna, kemudian jalankan blok berikut hanya dari Query Tool yang terhubung ke biosqlite_mg05.

```
DO $$
BEGIN
    IF current_database() <> 'biosqlite_mg05' THEN
        RAISE EXCEPTION 'Database tujuan tidak sesuai';
    END IF;

    EXECUTE 'DROP SCHEMA public RESTRICT';
END
$$;
```

Hasil normal pada tab **Messages** adalah DO. RESTRICT akan menolak penghapusan jika schema telah berisi objek. Jika muncul galat atau schema sudah berisi objek, hentikan proses dan hubungi asisten. Jangan mengganti RESTRICT dengan CASCADE. Proses pemulihan akan membuat schema public kembali.

4 Memulihkan Struktur dan Data melalui pgAdmin

Arsip BioSQLite_Mg05.v1.backup memuat struktur tabel, data, dan aturan relasi yang akan dipulihkan ke database biosqlite_mg05.

4.1 Memilih Arsip Backup

Pada **Object Explorer**, klik kanan `biosqlite_mg05`, lalu pilih **Restore**. Pastikan nama database tujuan sudah benar. Pada bagian **General**, lakukan pengaturan berikut.

1. Pilih **Custom or tar** atau **Custom** pada bagian **Format**.
2. Klik ikon folder pada **Filename**, kemudian pilih `BioSQLite_Mg05_v1.backup` yang telah disediakan.
3. Kosongkan **Role name**. Biarkan **Number of jobs** kosong atau isi dengan nilai 1.

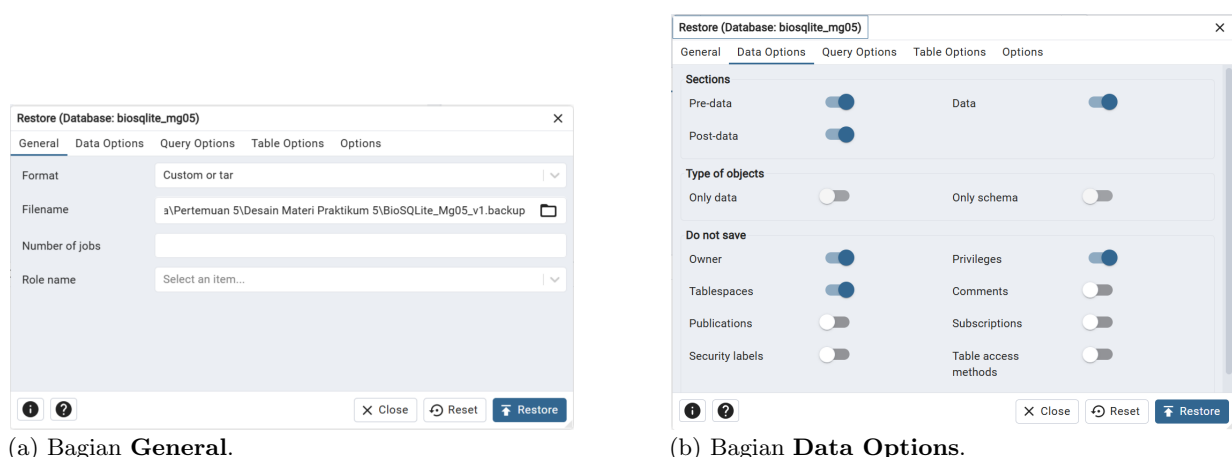
Pada bagian **Data Options**, cocokkan pengaturan dengan Tabel 2. Letak opsi dapat berbeda antar versi pgAdmin; cocokkan nama dan nilainya dengan tabel tersebut.

Tabel 2. Pengaturan pemulihan database BioSQLite.

Opsi	Pengaturan
Sections: Pre-data, Data, dan Post-data	Ketiganya Yes
Only data dan Only schema	Keduanya No
Do not save: Owner, Privileges, dan Tablespaces	Ketiganya Yes
Do not save: Comments	No
Include CREATE DATABASE	No
Clean before restore dan Include IF EXISTS	Keduanya No
Single transaction	Yes
Exit on error dan Verbose messages	Keduanya Yes
Disable triggers	No

Aktifkan **Pre-data**, **Data**, dan **Post-data** agar struktur, isi tabel, dan relasi dipulihkan. **Single transaction** membatalkan seluruh proses jika terjadi kegagalan. **Clean before restore** tidak digunakan karena restore dilakukan pada database baru.

Contoh pengaturan bagian **General** dan **Data Options** ditunjukkan pada Gambar 4.



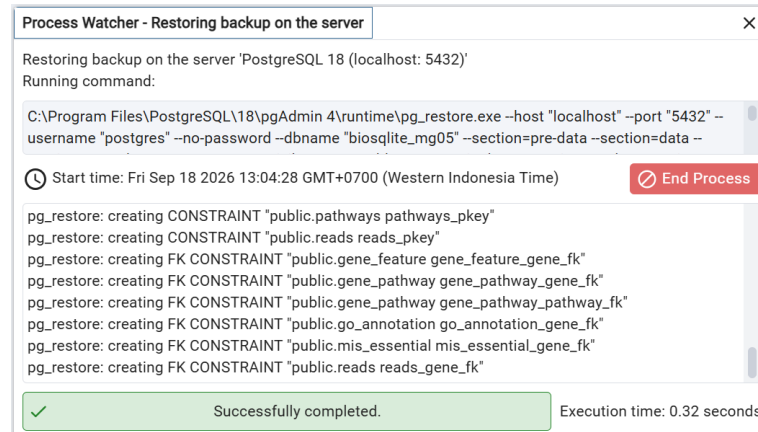
Gambar 4. Pengaturan arsip dan bagian restore.

4.2 Menjalankan dan Memeriksa Restore

Setelah seluruh opsi sesuai, klik **Restore** satu kali dan tunggu sampai notifikasi selesai. Klik **View Processes**, kemudian buka rincian proses restore terbaru dan lakukan pemeriksaan berikut.

1. Pastikan status proses menunjukkan **Successfully completed** atau **Finished**.
2. Periksa log dan pastikan terdapat pemrosesan data untuk ketujuh tabel serta pembuatan *foreign key constraint*.
3. Klik kanan database **biosqlite_mg05**, pilih **Refresh**, lalu buka **Schemas** → **public** → **Tables**. Pastikan ketujuh tabel terlihat.

Contoh proses restore yang berhasil ditunjukkan pada Gambar 5.



Gambar 5. Proses restore yang berhasil.

Catatan: Titik Pemeriksaan

Lanjutkan hanya jika database tujuan benar, proses restore berhasil, dan schema **public** memuat tujuh tabel.

5 Memeriksa Kelengkapan Hasil Restore

Buka kembali **Query Tool** dari **biosqlite_mg05**, lalu jalankan kedua query berikut secara terpisah. Lanjutkan hanya jika jumlah baris dan constraint sesuai dengan hasil yang diharapkan.

5.1 Memeriksa Jumlah Baris

Jalankan query berikut untuk menghitung seluruh baris pada ketujuh tabel.

```
SELECT 'gene' AS tabel, COUNT(*) AS jumlah_baris
FROM public.gene
UNION ALL
SELECT 'gene_feature', COUNT(*)
FROM public.gene_feature
UNION ALL
SELECT 'gene_pathway', COUNT(*)
FROM public.gene_pathway
UNION ALL
SELECT 'go_annotation', COUNT(*)
FROM public.go_annotation
UNION ALL
SELECT 'mis_essential', COUNT(*)
FROM public.mis_essential
UNION ALL
```

```
SELECT 'pathways', COUNT(*)
FROM public.pathways
UNION ALL
SELECT 'reads', COUNT(*)
FROM public.reads
ORDER BY tabel;
```

Cocokkan hasilnya dengan jumlah pada Tabel 1. UNION ALL hanya menyusun tujuh hasil hitungan secara vertikal; operasi ini berbeda dari JOIN yang mencocokkan baris antartabel.

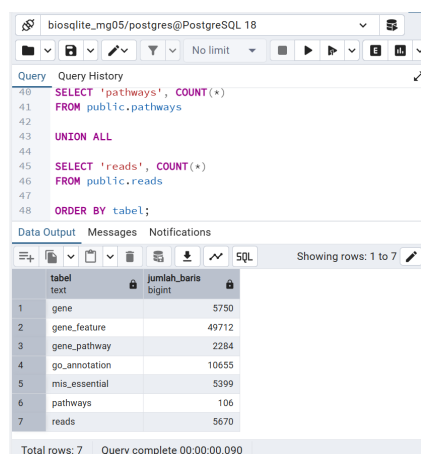
5.2 Memeriksa Primary Key dan Foreign Key

Query berikut membaca katalog internal PostgreSQL untuk menghitung *primary key* dan *foreign key* pada schema public.

```
SELECT
  CASE c.contype
    WHEN 'p' THEN 'PRIMARY KEY'
    WHEN 'f' THEN 'FOREIGN KEY'
  END AS jenis_constraint,
  COUNT(*) AS jumlah,
  BOOL_AND(c.convalidated) AS seluruhnya_tervalidasi
FROM pg_catalog.pg_constraint AS c
JOIN pg_catalog.pg_class AS t
  ON t.oid = c.conrelid
JOIN pg_catalog.pg_namespace AS n
  ON n.oid = t.relnamespace
WHERE n.nspname = 'public'
  AND c.contype IN ('p', 'f')
GROUP BY c.contype
ORDER BY jenis_constraint;
```

Hasil yang diharapkan adalah 6 *foreign key* dan 7 *primary key*; keduanya memiliki nilai validasi true. Query ini membaca katalog internal PostgreSQL untuk memastikan constraint hasil restore sudah terbentuk dan tervalidasi. *Primary key* menjaga keunikan identitas baris, sedangkan *foreign key* menjaga keabsahan rujukan antartabel.

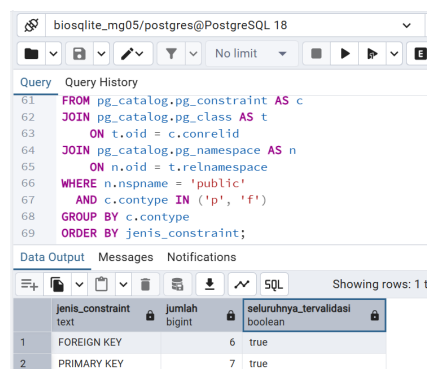
Contoh hasil kedua pemeriksaan ditunjukkan pada Gambar 6.



tabel	jumlah_baris
gene	5750
gene_feature	49712
gene_pathway	2284
go_annotation	10655
mis_essential	5399
pathways	106
reads	5670

Total rows: 7 Query complete 00:00:00.090

(a) Jumlah baris pada ketujuh tabel.



jenis_constraint	jumlah	seluruhnya_tervalidasi
FOREIGN KEY	6	true
PRIMARY KEY	7	true

Showing rows: 1 to 2

(b) Primary key dan foreign key tervalidasi.

Gambar 6. Pemeriksaan kelengkapan data dan constraint sebelum latihan query dimulai.

6 Menggunakan Query Tool dan Membaca Query

Mulai bagian ini, seluruh kegiatan menggunakan **SELECT** untuk membaca data. **JOIN** menggabungkan baris berdasarkan hubungan antartabel tanpa membuat tabel baru atau mengubah data yang tersimpan.

Gunakan pola kerja berikut setiap kali menjalankan latihan.

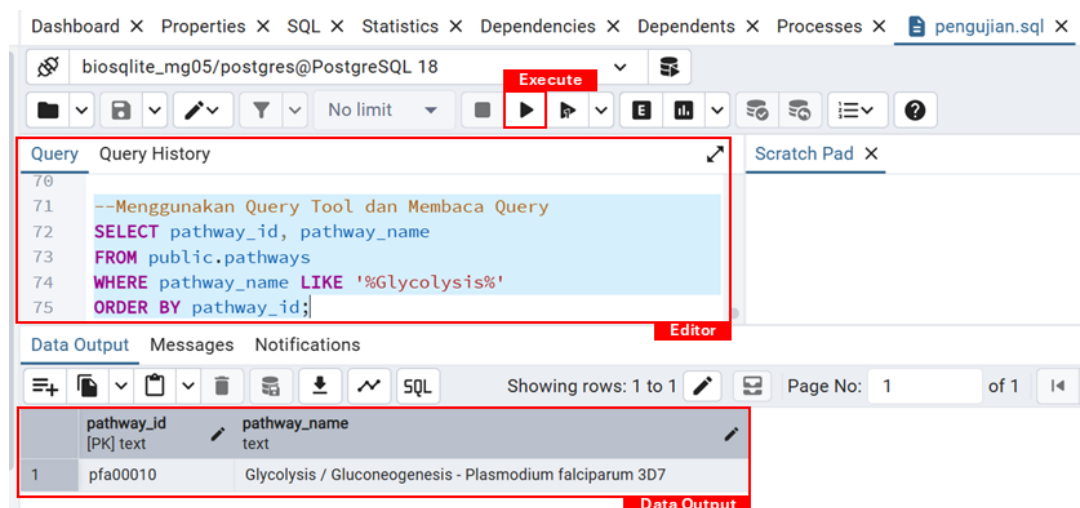
1. Buka **Query Tool** dari `biosqlite_mg05`, lalu simpan berkas dengan nama `NIM_Nama.Praktikum05.sql`.
2. Masukkan satu query. Jika editor berisi beberapa query, sorot seluruh query yang akan dijalankan, termasuk tanda titik koma.
3. Klik **Execute**. Baca hasil pada **Data Output** dan pesan kesalahan pada **Messages**. Pastikan batas baris pada toolbar tidak membatasi hasil.
4. Gunakan query `COUNT(*)` untuk menghitung seluruh hasil. `LIMIT 20` hanya menampilkan cuplikan.
5. Tambahkan komentar `--` berisi prediksi, jumlah hasil, dan interpretasi. Simpan skrip setelah setiap latihan.

Jalankan query berikut untuk mencari pathway yang namanya memuat kata *Glycolysis*.

```
SELECT pathway_id, pathway_name
FROM public.pathways
WHERE pathway_name LIKE '%Glycolysis%'
ORDER BY pathway_id;
```

Pada paket ini, query menemukan `pfa00010`, yaitu pathway *glycolysis/gluconeogenesis*. `LIKE` dengan `%` mencari potongan teks; perbedaan huruf besar dan kecil dapat memengaruhi hasil. Salin identifier persis sebagaimana tersimpan dalam database.

Contoh tampilan Query Tool setelah query filtering dan sorting dijalankan ditunjukkan pada Gambar 7.



Gambar 7. Query Tool saat menjalankan query filtering dan sorting.

Gunakan kosakata berikut untuk membaca query **JOIN** pada latihan selanjutnya.

Tabel 3. Kosakata dasar untuk membaca query JOIN.

Bentuk	Cara membacanya
FROM public.reads AS r	Mengambil baris dari reads ; r adalah alias sementara.
r.gene_id	Kolom gene_id dari tabel yang diberi alias r .
ON r.gene_id = m.gene_id	Memasangkan baris yang memiliki identifier sama.
WHERE ...	Menyaring baris berdasarkan kondisi.
ORDER BY ...	Mengurutkan hasil agar pemeriksaan dapat diulang.
NULL dan IS NULL	Nilai tidak tersedia; periksa dengan IS NULL , bukan = NULL .

Catatan: Panduan Sebelum JOIN

Sebelum menjalankan setiap **JOIN**, tentukan informasi yang dicari, tabel yang menyimpannya, kolom pencocokan, sisi yang harus dipertahankan, dan arti satu baris hasil.

7 INNER JOIN untuk Data yang Tersedia Bersama

7.1 Menggabungkan Data RNA-seq dan Mutagenesis

Pertanyaan pada latihan ini adalah: identifier mana yang memiliki data RNA-seq sekaligus skor mutagenesis? Informasi RNA-seq tersimpan dalam tabel **reads**, sedangkan hasil eksperimen mutagenesis tersimpan dalam tabel **mis_essential**. Kedua tabel menggunakan **gene_id** sebagai identifier yang unik.

Catatan: Prediksi Sebelum Menjalankan Query

Prediksi apakah jumlah pasangan hasil **INNER JOIN** dapat melebihi jumlah baris tabel **reads**. Jelaskan prediksi berdasarkan keunikan **gene_id** pada kedua tabel.

Jalankan query berikut untuk menampilkan dua nilai *raw count* RNA-seq, kategori mutabilitas, dan skor *fitness* bagi identifier yang tersedia pada kedua tabel.

```
SELECT
    r.gene_id,
    r.wt_1,
    r.ko_1,
    m.mutability,
    m.mutagenesis_fitness_score
FROM public.reads AS r
INNER JOIN public.mis_essential AS m
    ON r.gene_id = m.gene_id
ORDER BY r.gene_id
LIMIT 20;
```

Baca query mulai dari **FROM**, kemudian **INNER JOIN** dan **ON**, lalu kolom pada **SELECT**. Alias **r** dan **m** hanya berlaku di dalam query dan tidak mengubah nama tabel. **INNER JOIN** mempertahankan pasangan yang memenuhi kondisi **ON**, sedangkan **LIMIT 20** hanya membatasi cuplikan yang ditampilkan.

Periksa baris PF3D7_0100100. Nilai yang diharapkan dirangkum pada Tabel 4.

Tabel 4. Titik pemeriksaan hasil INNER JOIN untuk PF3D7_0100100.

Kolom	Nilai yang diharapkan
wt_1	224
ko_1	227
mutability	Mutable in CDS
mutagenesis_fitness_score	-1.706

7.2 Memeriksa Jumlah Pasangan dan Interpretasi

Query dengan LIMIT 20 belum menunjukkan jumlah seluruh pasangan. Jalankan query berikut secara terpisah.

```
SELECT COUNT(*) AS jumlah_pasangan
FROM public.reads AS r
INNER JOIN public.mis_essential AS m
ON r.gene_id = m.gene_id;
```

Hasil yang diharapkan adalah `jumlah_pasangan = 5399`. Karena kolom pencocokan unik pada kedua tabel, setiap baris hasil mewakili satu identifier yang tersedia pada kedua sumber. Identifier yang tidak memiliki pasangan pada salah satu tabel tidak ditampilkan oleh INNER JOIN. Dengan demikian, jumlah hasil tidak dapat melebihi jumlah baris pada salah satu tabel yang digabungkan. Contoh hasil query dan hitungan seluruh pasangan ditunjukkan pada Gambar 8.

gene_id	wt_1	ko_1	mutability	mutagenesis_fitness_score
PF3D7_0100100	224	227	Mutable in CDS	-1.706
PF3D7_0100200	11	5	Non - Mutable in CDS	-2.624
PF3D7_0100300	4	1	Non - Mutable in CDS	-2.461
PF3D7_0100400	21	7	Non - Mutable in CDS	-1.854

(a) Cuplikan 20 baris hasil.

jumlah_pasangan
5399

(b) Hitungan seluruh pasangan.

Gambar 8. INNER JOIN menggabungkan *raw count* RNA-seq dan skor mutagenesis untuk identifier yang cocok.

Nilai pada `wt_1` dan `ko_1` merupakan *raw count* yang belum dinormalisasi. Perbedaan kedua nilai pada satu ulangan belum cukup untuk menyimpulkan perubahan ekspresi yang signifikan. Latihan ini menyusun informasi dari dua tabel dan tidak melakukan analisis ekspresi diferensial. Skor mutagenesis juga tidak langsung diberi ambang “esensial” tanpa kriteria dan rujukan eksperimen yang sesuai.

Tugas: Latihan Singkat

Ubah proyeksi **SELECT** agar menampilkan **wt_2** dan **ko_2**. Jalankan kembali query, lalu jelaskan apakah perubahan kolom keluaran tersebut mengubah jumlah pasangan.

8 Menghubungkan Identifier dengan Nama Pathway

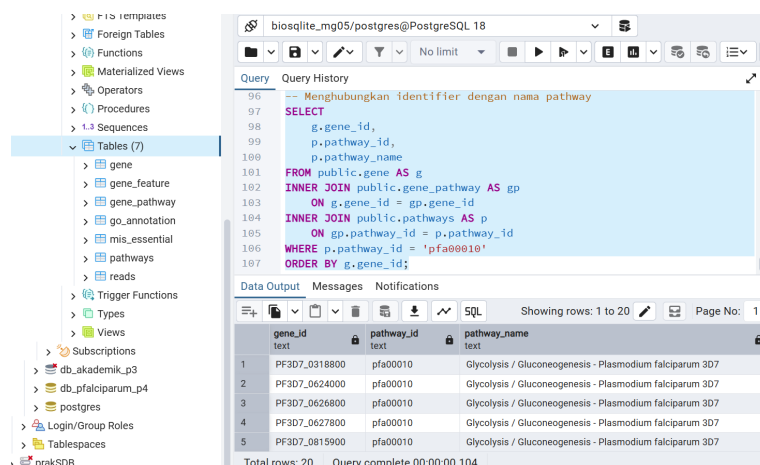
8.1 Menelusuri Hubungan Tiga Tabel

Pertanyaan pada latihan ini adalah: identifier apa saja yang tercatat pada pathway *glycolysis/gluconeogenesis*? Tabel **gene** tidak menyimpan nama pathway, sehingga hubungan perlu ditelusuri dari **gene** ke **gene_pathway**, lalu ke **pathways**. Sebelum menjalankan query, periksa kembali hubungan tersebut pada ERD.

```
SELECT
  g.gene_id,
  p.pathway_id,
  p.pathway_name
FROM public.gene AS g
INNER JOIN public.gene_pathway AS gp
  ON g.gene_id = gp.gene_id
INNER JOIN public.pathways AS p
  ON gp.pathway_id = p.pathway_id
WHERE p.pathway_id = 'pfa00010'
ORDER BY g.gene_id;
```

Kondisi **ON** pertama memasangkan **gene** dengan **gene_pathway** melalui **gene_id**. Kondisi **ON** kedua memasangkan **gene_pathway** dengan **pathways** melalui **pathway_id**. Klausa **WHERE** kemudian memilih pathway **pfa00010**.

Hasil yang diharapkan berisi 20 baris dan seluruhnya mempunyai **pathway_id** bernilai **pfa00010**. Satu baris hasil bermakna satu pasangan identifier–pathway, bukan satu gen unik. Satu identifier dapat muncul pada beberapa pathway yang berbeda. Untuk memeriksa cakupan penuh, hapus sementara klausa **WHERE**; hasilnya seharusnya memuat 2284 pasangan identifier–pathway, lalu kembalikan klausa tersebut. Contoh hasil penelusuran tiga tabel ditunjukkan pada Gambar 9.



The screenshot shows a PostgreSQL query editor interface. On the left, a tree view displays the database structure, including tables like **gene**, **gene_pathway**, and **pathways**. The main window shows a SQL query that joins the **gene**, **gene_pathway**, and **pathways** tables, filtering for **pfa00010**. Below the query, the 'Data Output' tab displays the results of the query.

	gene_id	pathway_id	pathway_name
1	PF3D7_0318800	pfa00010	Glycolysis / Gluconeogenesis - Plasmodium falciparum 3D7
2	PF3D7_0624000	pfa00010	Glycolysis / Gluconeogenesis - Plasmodium falciparum 3D7
3	PF3D7_0626800	pfa00010	Glycolysis / Gluconeogenesis - Plasmodium falciparum 3D7
4	PF3D7_0627800	pfa00010	Glycolysis / Gluconeogenesis - Plasmodium falciparum 3D7
5	PF3D7_0815900	pfa00010	Glycolysis / Gluconeogenesis - Plasmodium falciparum 3D7
Total rows: 20 Query complete 00:00:00.104			

Gambar 9. Tabel penghubung menelusuri identifier ke nama pathway.

Primary key gabungan pada **gene_pathway** mencegah pasangan **gene_id** dan **pathway_id** yang

sama disimpan lebih dari satu kali, tetapi tetap mengizinkan satu identifier terhubung dengan beberapa pathway.

8.2 Memeriksa Jumlah Identifier yang Berbeda

Jalankan query berikut untuk membandingkan jumlah seluruh pasangan dengan jumlah identifier yang berbeda.

```
SELECT
    COUNT(*) AS jumlah_pasangan,
    COUNT(DISTINCT gene_id) AS identifier_berbeda
FROM public.gene_pathway;
```

Titik pemeriksaan: hasil query adalah `jumlah_pasangan = 2284` dan `identifier_berbeda = 1232`. `COUNT(*)` menghitung seluruh pasangan identifier–pathway, sedangkan `COUNT(DISTINCT gene_id)` menghitung nilai identifier yang berbeda. Jumlah identifier berbeda tidak otomatis menyatakan jumlah gen biologis unik lintas seluruh sumber data.

Tugas: Penerapan Pola Relasi

Gunakan analogi mahasiswa–pengambilan mata kuliah–mata kuliah. Tentukan tabel yang berperan seperti `gene_pathway`, dua kolom penyusun *primary key* gabungan, dan cara menerapkan pola query yang sama pada kasus tersebut.

9 LEFT JOIN dan RIGHT JOIN untuk Mempertahankan Katalog

9.1 LEFT JOIN dan Identifier tanpa Pasangan

Pertanyaan pada latihan ini adalah: bagaimana menampilkan seluruh identifier dalam katalog `gene`, termasuk yang tidak memiliki baris RNA-seq? `INNER JOIN` tidak menampilkan identifier tanpa pasangan, sedangkan `LEFT JOIN` mempertahankan seluruh baris tabel di sebelah kiri.

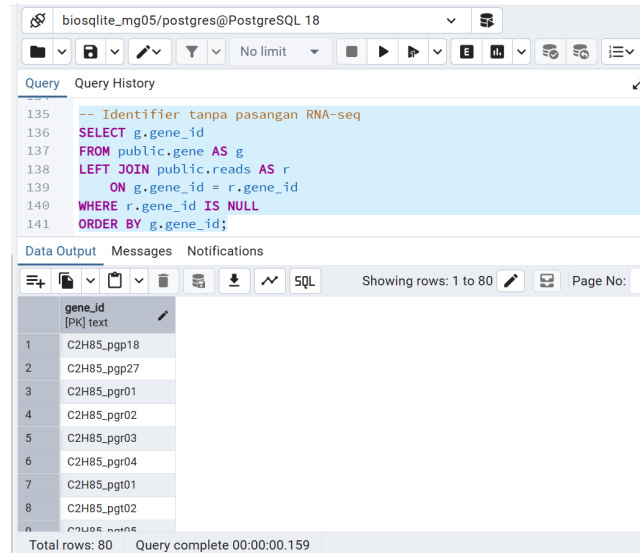
```
SELECT
    g.gene_id,
    r.wt_1,
    r.ko_1
FROM public.gene AS g
LEFT JOIN public.reads AS r
    ON g.gene_id = r.gene_id
ORDER BY g.gene_id;
```

Prediksi jumlah baris sebelum menjalankan query. Hasil yang diharapkan adalah 5750 baris, sama dengan jumlah identifier pada tabel `gene`. Kolom `reads` bernilai `NULL` jika pasangan tidak ditemukan; ini berbeda dari nilai hitungan 0 yang memang tersimpan pada tabel `reads`.

Gunakan query berikut untuk mengambil hanya identifier yang tidak mempunyai pasangan pada `reads`.

```
SELECT g.gene_id
FROM public.gene AS g
LEFT JOIN public.reads AS r
    ON g.gene_id = r.gene_id
WHERE r.gene_id IS NULL
ORDER BY g.gene_id;
```

Hasil yang diharapkan adalah 80 identifier. Kolom `r.gene_id` digunakan untuk mendeteksi pasangan yang hilang karena kolom tersebut merupakan primary key dan tidak bernilai NULL pada baris asli tabel `reads`. Pola `LEFT JOIN` yang diikuti pemeriksaan `IS NULL` pada sisi kanan sering disebut *anti join*. Contoh hasilnya ditunjukkan pada Gambar 10.



Gambar 10. Identifier dalam katalog yang belum mempunyai pasangan RNA-seq.

9.2 Menyatakan Tujuan yang Sama dengan `RIGHT JOIN`

Tujuan query pertama dapat dinyatakan dengan `RIGHT JOIN` setelah posisi kedua tabel ditukar.

```

SELECT
  g.gene_id,
  r.wt_1,
  r.ko_1
FROM public.reads AS r
RIGHT JOIN public.gene AS g
ON r.gene_id = g.gene_id
ORDER BY g.gene_id;

```

`RIGHT JOIN` mempertahankan seluruh baris tabel di sebelah kanan, yaitu `gene`. Jalankan query `LEFT JOIN` dan `RIGHT JOIN` secara terpisah, lalu pastikan keduanya menghasilkan 5750 baris dengan urutan `gene_id` yang sama.

Tugas: Interpretasi Hasil Anti Join

Jelaskan mengapa hasil 80 identifier tidak mendukung pernyataan “80 gen tidak diekspresikan”. Tuliskan kesimpulan yang hanya menyatakan cakupan tabel, kondisi pencocokan, dan hasil yang ditemukan.

10 Menempatkan Filter pada `ON` atau `WHERE`

Letak kondisi pada query menentukan baris yang dipertahankan. Bandingkan dua pertanyaan berikut: A menampilkan seluruh katalog identifier dan hanya menyertakan data RNA-seq jika `wt_1 > 0`, sedangkan B menampilkan hanya identifier yang mempunyai `wt_1 > 0`. Perbedaan tujuan ini menentukan apakah syarat tambahan diletakkan pada `ON` atau `WHERE`.

Query A menempatkan syarat tambahan pada ON.

```
SELECT
    g.gene_id,
    r.wt_1
FROM public.gene AS g
LEFT JOIN public.reads AS r
    ON g.gene_id = r.gene_id
    AND r.wt_1 > 0
ORDER BY g.gene_id;
```

Query B menempatkan syarat pada WHERE.

```
SELECT
    g.gene_id,
    r.wt_1
FROM public.gene AS g
LEFT JOIN public.reads AS r
    ON g.gene_id = r.gene_id
WHERE r.wt_1 > 0
ORDER BY g.gene_id;
```

Prediksi query mana yang mempertahankan seluruh katalog, lalu jalankan A dan B secara terpisah. Hasil yang diharapkan adalah A = 5750 baris dan B = 5291 baris. Pada A, nilai `wt_1` menjadi NULL ketika pasangan `reads` tidak ada atau ketika pasangannya gagal memenuhi syarat `wt_1 > 0`. Pada B, `WHERE` membuang baris yang tidak memenuhi syarat, termasuk nilai NULL.

Untuk membedakan nilai nol dari data yang hilang, hitung identifier pada `reads` yang memiliki `wt_1` bernilai nol.

```
SELECT COUNT(*) AS identifier_dengan_wt1_nol
FROM public.reads
WHERE wt_1 = 0;
```

Hasil yang diharapkan adalah 379. Bersama 80 identifier tanpa baris `reads`, terdapat 459 baris dengan `wt_1` bernilai NULL pada hasil A. Perubahan kondisi query dapat mengubah makna NULL pada hasil meskipun isi tabel tidak berubah.

Catatan: Titik Pemeriksaan Pemahaman

Lengkapi pernyataan berikut: “Jika tujuan saya mempertahankan seluruh katalog, saya menempatkan syarat tambahan pasangan pada ... karena ...” Kemudian buat satu pertanyaan menggunakan `ko_1`, tentukan letak filternya pada `ON` atau `WHERE`, dan jalankan query untuk memeriksa prediksi.

11 FULL OUTER JOIN untuk Membandingkan Cakupan Sumber

Pertanyaan pada latihan ini adalah: bagaimana cakupan identifier RNA-seq dibandingkan dengan anotasi *pathway*? Baris yang cocok maupun baris yang hanya tersedia pada salah satu sumber perlu dipertahankan. Kebutuhan tersebut dapat dipenuhi menggunakan `FULL OUTER JOIN`.

```
SELECT
    r.gene_id AS id_rnaseq,
    gp.gene_id AS id_pathway,
    gp.pathway_id
```

```
FROM public.reads AS r
FULL OUTER JOIN public.gene_pathway AS gp
  ON r.gene_id = gp.gene_id
ORDER BY r.gene_id, gp.gene_id, gp.pathway_id;
```

Baca kedua kolom identifier secara terpisah. Jika `id_rnaseq` dan `id_pathway` terisi, baris tersebut mempunyai pasangan pada kedua sumber. Jika `id_pathway` bernilai `NULL`, identifier RNA-seq tidak mempunyai pasangan *pathway*. Jika `id_rnaseq` bernilai `NULL`, pasangan *pathway* tidak mempunyai identifier yang cocok pada `reads`. Satu identifier yang mempunyai beberapa *pathway* menghasilkan beberapa baris.

Gunakan ekspresi `CASE WHEN` untuk mengelompokkan hasil berdasarkan keberadaan identifier pada kedua sumber.

```
SELECT
  COUNT(*) AS seluruh_baris,
  COUNT(
    CASE
      WHEN r.gene_id IS NOT NULL
        AND gp.gene_id IS NOT NULL
      THEN 1
    END
  ) AS baris_cocok,
  COUNT(
    CASE
      WHEN gp.gene_id IS NULL
      THEN 1
    END
  ) AS hanya_rnaseq,
  COUNT(
    CASE
      WHEN r.gene_id IS NULL
      THEN 1
    END
  ) AS hanya_pathway
FROM public.reads AS r
FULL OUTER JOIN public.gene_pathway AS gp
  ON r.gene_id = gp.gene_id;
```

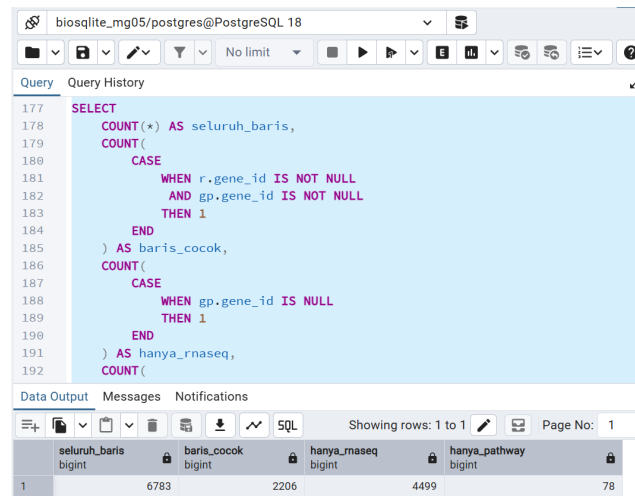
`CASE WHEN` memberi nilai 1 hanya pada kondisi yang terpenuhi, lalu `COUNT` menghitung nilai yang tidak `NULL`. Cocokkan hasil query dengan Tabel 5.

Tabel 5. Ringkasan cakupan hasil `FULL OUTER JOIN`.

Ukuran	Hasil	Satuan
Seluruh baris	6.783	Baris hasil <code>FULL OUTER JOIN</code>
Baris cocok	2.206	Pasangan identifier- <i>pathway</i>
Hanya RNA-seq	4.499	Identifier tanpa pasangan <i>pathway</i>
Hanya <i>pathway</i>	78	Pasangan <i>pathway</i> tanpa pasangan <code>reads</code>

Pastikan jumlah ketiga kelompok sama dengan seluruh baris: $2.206 + 4.499 + 78 = 6.783$. Sebanyak 78 baris pada kelompok hanya *pathway* berasal dari 61 identifier berbeda. Oleh karena satu identifier dapat mempunyai beberapa *pathway*, angka 78 tidak boleh ditafsirkan sebagai jumlah

gen. Contoh ringkasan hasil FULL OUTER JOIN ditunjukkan pada Gambar 11.



The screenshot shows a PostgreSQL query editor with the following SQL query:

```
SELECT
  COUNT(*) AS seluruh_baris,
  COUNT(
    CASE
      WHEN r.gene_id IS NOT NULL
      AND gp.gene_id IS NOT NULL
      THEN 1
    END
  ) AS baris_cocok,
  COUNT(
    CASE
      WHEN gp.gene_id IS NULL
      THEN 1
    END
  ) AS hanya_rnaseq,
  COUNT(
    CASE
      WHEN r.gene_id IS NULL
      THEN 1
    END
  ) AS hanya_pathway
```

The results table shows the following data:

	seluruh_baris bigint	baris_cocok bigint	hanya_rnaseq bigint	hanya_pathway bigint
1	6783	2206	4499	78

Gambar 11. FULL OUTER JOIN memperlihatkan kecocokan dan kesenjangan cakupan dari kedua sumber.

Tugas: Refleksi FULL OUTER JOIN

Mengapa FULL OUTER JOIN tidak selalu menghasilkan lebih banyak baris daripada LEFT JOIN? Jika semua baris sisi kanan memiliki pasangan di kiri, tidak ada baris tambahan dari sisi kanan.

12 CROSS JOIN dan Self Join

CROSS JOIN untuk ruang kombinasi. Pertanyaan pada latihan ini adalah: berapa kombinasi identifier-*pathway* yang mungkin jika setiap identifier pada *gene* dipasangkan dengan setiap *pathway*? CROSS JOIN membentuk seluruh kombinasi baris dari kedua tabel tanpa menggunakan kondisi ON.

```
SELECT COUNT(*) AS kombinasi_mungkin
FROM public.gene AS g
CROSS JOIN public.pathways AS p;
```

Prediksi hasilnya dengan mengalikan jumlah baris kedua tabel. Sebanyak 5750 identifier dikombinasikan dengan 106 *pathway*, sehingga query menghasilkan 609500 kombinasi. Seluruh kombinasi tidak perlu ditampilkan pada layar.

Bandingkan angka tersebut dengan 2284 pasangan yang tersimpan pada *gene_pathway*. Ruang kombinasi dapat dipakai untuk menyusun calon anotasi yang perlu diperiksa, tetapi bukan bukti 609500 hubungan biologis.

Self join untuk pasangan yang berbagi pathway. Pertanyaan berikutnya adalah: identifier mana yang sama-sama tercatat pada *pathway* pfa00010? *Self join* menggunakan tabel yang sama lebih dari satu kali dengan alias berbeda; *self join* bukan kata kunci SQL tersendiri.

```
SELECT
  a.gene_id AS identifier_a,
  b.gene_id AS identifier_b,
  a.pathway_id
```

```
FROM public.gene_pathway AS a
INNER JOIN public.gene_pathway AS b
  ON a.pathway_id = b.pathway_id
  AND a.gene_id < b.gene_id
WHERE a.pathway_id = 'pfa00010'
ORDER BY a.gene_id, b.gene_id;
```

Kondisi `a.gene_id < b.gene_id` membuang pasangan diri sendiri dan mencegah duplikasi A–B/B–A. Operator `<` membandingkan teks identifier untuk memilih satu urutan, bukan menunjukkan urutan biologis.

Sebanyak 20 identifier tercatat pada `pfa00010`. Jumlah pasangan unik yang dihasilkan adalah

$$\frac{20 \times 19}{2} = 190.$$

Berbagi *pathway* tidak membuktikan bahwa kedua identifier berinteraksi secara fisik.

Ringkasan pemilihan JOIN. Gunakan tujuan analisis untuk menentukan jenis penggabungan seperti dirangkum pada Tabel 6.

Tabel 6. Ringkasan pemilihan jenis *join*.

Tujuan	Pilihan
Hanya mempertahankan pasangan yang memenuhi ON	INNER JOIN
Mempertahankan seluruh sisi kiri	LEFT JOIN
Mempertahankan seluruh sisi kanan	RIGHT JOIN
Mempertahankan kedua sisi, termasuk yang tidak cocok	FULL OUTER JOIN
Membentuk seluruh kombinasi baris kedua tabel	CROSS JOIN
Membandingkan baris dalam tabel yang sama	<i>Self join</i> dengan dua alias

Klausula `ON` menyatakan syarat pasangan secara eksplisit. `USING (gene_id)` dapat digunakan jika nama kolom pencocokan sama, tetapi latihan ini menggunakan `ON` agar asal setiap kolom terlihat. Hindari `NATURAL JOIN` pada latihan karena perintah tersebut memilih semua nama kolom yang sama secara otomatis; perubahan struktur tabel dapat mengubah aturan pencocokan tanpa terlihat pada query.

13 Merangkum Hasil dengan GROUP BY dan HAVING

Menghitung identifier pada setiap pathway. Pertanyaan pada latihan ini adalah: berapa identifier yang tercatat pada setiap *pathway*? Gunakan fungsi agregasi `COUNT` dan bentuk kelompok berdasarkan identifier serta nama *pathway*.

```
SELECT
  p.pathway_id,
  p.pathway_name,
  COUNT(gp.gene_id) AS jumlah_identifier
FROM public.pathways AS p
LEFT JOIN public.gene_pathway AS gp
  ON p.pathway_id = gp.pathway_id
GROUP BY p.pathway_id, p.pathway_name
ORDER BY jumlah_identifier DESC, p.pathway_id;
```

Setelah agregasi, satu baris hasil mewakili satu *pathway*, bukan satu pasangan identifier–*pathway*. Query menghasilkan 106 kelompok. Periksa baris *pfa00010*; nilai *jumlah_identifier* yang diharapkan adalah 20.

Menyaring kelompok dengan HAVING. Pertanyaan berikutnya adalah: *pathway* mana yang mempunyai setidaknya 20 identifier? Tambahkan **HAVING** setelah **GROUP BY** dan sebelum **ORDER BY**.

```
SELECT
  p.pathway_id,
  p.pathway_name,
  COUNT(gp.gene_id) AS jumlah_identifier
FROM public.pathways AS p
LEFT JOIN public.gene_pathway AS gp
  ON p.pathway_id = gp.pathway_id
GROUP BY p.pathway_id, p.pathway_name
HAVING COUNT(gp.gene_id) >= 20
ORDER BY jumlah_identifier DESC, p.pathway_id;
```

Catatan: Titik Pemeriksaan Pemahaman

Sebelum membaca penjelasan berikut, jelaskan mengapa kondisi agregat tersebut ditempatkan pada **HAVING**, bukan pada **WHERE**.

Ambang 20 dipilih untuk latihan SQL dan bukan batas signifikansi atau kepentingan biologis. **WHERE** menyaring baris sebelum agregasi, sedangkan **HAVING** menyaring kelompok setelah agregasi. Karena itu, kondisi **COUNT(gp.gene_id) >= 20** diletakkan pada **HAVING**. Pada snapshot praktikum, kondisi tersebut mempertahankan 29 kelompok *pathway*; *pfa00010* termasuk di dalamnya dengan *jumlah_identifier* sebesar 20.

Mengapa COUNT menggunakan kolom tabel anak. **LEFT JOIN** tetap mempertahankan *pathway* yang tidak mempunyai pasangan. Pada kondisi tersebut, **COUNT(*)** menghitung satu baris hasil penggabungan, sedangkan **COUNT(gp.gene_id)** menghasilkan nol karena nilai kolom pada sisi anak adalah **NULL**. Dalam snapshot praktikum ini, semua *pathway* mempunyai pasangan. Kesamaan hasil kedua bentuk **COUNT** pada data ini tidak berarti keduanya selalu setara.

Merangkum raw count untuk orientasi data. Gunakan fungsi agregasi berikut untuk mengenali skala nilai pada kolom *wt_1*.

```
SELECT
  MIN(wt_1) AS count_minimum,
  MAX(wt_1) AS count_maksimum,
  AVG(wt_1) AS count_rerata
FROM public.reads;
```

MIN, **MAX**, dan **AVG** merangkum nilai minimum, maksimum, dan rerata pada kolom numerik. Ringkasan ini membantu mengenali skala *raw count*, tetapi rerata antargen bukan ukuran perubahan ekspresi **WT–KO** dan tidak menggantikan normalisasi atau pengujian statistik.

Contoh hasil query **GROUP BY** dengan **HAVING** ditunjukkan pada Gambar 12.

Query

```

263 --Menyaring Kelompok dengan HAVING
264 SELECT
265     p.pathway_id,
266     p.pathway_name,
267     COUNT(gp.gene_id) AS jumlah_identifier
268 FROM public.pathways AS p
269 LEFT JOIN public.gene_pathway AS gp
270 ON p.pathway_id = gp.pathway_id
271 GROUP BY p.pathway_id, p.pathway_name
272 HAVING COUNT(gp.gene_id) >= 20
273 ORDER BY jumlah_identifier DESC, p.pathway_id;

```

Data Output Messages Notifications

Showing rows: 1 to 29 Page No: 1

	pathway_id [PK] text	pathway_name text	jumlah_identifier bigint
24	pfa01212	Fatty acid metabolism - Plasmodium falciparum 3D7	24
25	pfa03020	RNA polymerase - Plasmodium falciparum 3D7	24
26	pfa04144	Endocytosis - Plasmodium falciparum 3D7	24
27	pfa00564	Glycerophospholipid metabolism - Plasmodium falciparum 3D7	22
28	pfa00010	Glycolysis / Gluconeogenesis - Plasmodium falciparum 3D7	20
29	pfa03430	Mismatch repair - Plasmodium falciparum 3D7	20

Total rows: 29 Query complete 00:00:00.157

Gambar 12. Agregasi mengubah satuan keluaran menjadi satu baris per *pathway*.

14 Subquery untuk Pertanyaan Bertingkat

Subquery yang menghasilkan daftar. Pertanyaan pada latihan ini adalah: dari identifier yang tercatat pada *pathway* pfa00010, identifier mana yang mempunyai baris RNA-seq? Pilih lebih dahulu identifier *pathway* melalui *subquery*, lalu gunakan daftar tersebut sebagai syarat pencarian pada *reads*.

```

SELECT
    r.gene_id,
    r.wt_1,
    r.ko_1
FROM public.reads AS r
WHERE r.gene_id IN (
    SELECT gp.gene_id
    FROM public.gene_pathway AS gp
    WHERE gp.pathway_id = 'pfa00010'
)
ORDER BY r.gene_id;

```

Jalankan lebih dahulu perintah **SELECT** di dalam tanda kurung. Bagian tersebut merupakan *subquery* yang menghasilkan daftar 20 identifier pada pfa00010. Setelah seluruh query dijalankan, operator **IN** memeriksa apakah **gene_id** pada *reads* terdapat dalam daftar tersebut. Karena **gene_id** merupakan primary key pada *reads*, setiap identifier paling banyak menghasilkan satu baris dari query luar. Nilai berulang pada hasil *subquery* tidak menggandakan baris query luar.

Catatan: Perbandingan dengan INNER JOIN

Tulis versi **INNER JOIN** antara *reads* dan *gene_pathway* dengan filter **pathway_id = 'pfa00010'**. Urutkan berdasarkan **gene_id**, kemudian bandingkan daftar identifier dan jumlah barisnya dengan hasil query yang menggunakan **IN**.

Subquery yang menghasilkan satu nilai. *Subquery* juga dapat menghasilkan satu nilai yang digunakan sebagai pembanding oleh query luar. Contoh berikut menghitung rerata **wt_1**, kemudian menampilkan 20 baris dengan nilai **wt_1** tertinggi di atas rerata tersebut.

```

SELECT

```

```

    gene_id,
    wt_1
FROM public.reads
WHERE wt_1 > (
    SELECT AVG(wt_1)
    FROM public.reads
)
ORDER BY wt_1 DESC, gene_id
LIMIT 20;

```

Berbeda dari contoh IN yang memerlukan daftar, bentuk ini memerlukan *subquery* bernilai tunggal. Nilai pembanding dihitung melalui *subquery* agar selalu berasal dari isi tabel yang sedang digunakan.

LIMIT 20 hanya membatasi cuplikan yang ditampilkan dan bukan jumlah seluruh identifier dengan wt_1 di atas rerata. Hasil tersebut juga tidak otomatis menunjukkan peningkatan ekspresi karena belum melibatkan normalisasi, perbandingan kelompok, atau pengujian statistik.

Catatan: Titik Pemeriksaan Pemahaman

Untuk setiap subquery, identifikasi pertanyaan bagian dalam, bentuk hasilnya (daftar atau satu nilai), dan cara hasil tersebut digunakan oleh query luar.

Tugas: Latihan Mandiri Subquery

Temukan identifier yang memiliki anotasi GO menggunakan pola WHERE gene_id IN (SELECT gene_id FROM public.go_annotation). Mulailah dari tabel gene. Prediksi jumlah baris sebelum menjalankan query, lalu bandingkan dengan 3.472 identifier berbeda yang mempunyai anotasi. Jelaskan mengapa banyak baris anotasi untuk identifier yang sama pada go_annotation tidak menggandakan baris dari tabel gene ketika operator IN digunakan.

15 Memeriksa Pertambahan Baris pada JOIN Anotasi

15.1 Menggabungkan Pathway dengan Anotasi GO

Pertanyaan pada latihan ini adalah: anotasi GO apa saja yang tersedia untuk identifier pada pfa00010? Satu identifier dapat mempunyai beberapa rekaman GO, sehingga penggabungan berikut tidak selalu menghasilkan satu baris per identifier.

```

SELECT
    gp.gene_id,
    gp.pathway_id,
    a.go_id,
    a.evidence_code,
    a.gene_product_name
FROM public.gene_pathway AS gp
INNER JOIN public.go_annotation AS a
    ON gp.gene_id = a.gene_id
WHERE gp.pathway_id = 'pfa00010'
ORDER BY gp.gene_id, a.go_id, a.annotation_id;

```

Satu baris hasil mewakili satu rekaman anotasi GO untuk satu identifier yang tercatat pada

pfa00010. Kolom `gene_product_name` menyimpan nama produk gen dan bukan nama istilah GO. `go_id` merupakan pengenal istilah GO, sedangkan `evidence_code` menunjukkan jenis bukti anotasi. Penafsiran terperinci terhadap istilah GO atau kode bukti memerlukan rujukan ontologi dan berada di luar tujuan query ini.

Hitung seluruh baris hasil penggabungan dan jumlah identifier berbeda.

```
SELECT
    COUNT(*) AS jumlah_baris,
    COUNT(DISTINCT gp.gene_id) AS identifier_berbeda
FROM public.gene_pathway AS gp
INNER JOIN public.go_annotation AS a
    ON gp.gene_id = a.gene_id
WHERE gp.pathway_id = 'pfa00010';
```

Titik pemeriksaan: hasil query adalah 103 baris dan 19 identifier berbeda. Sebelumnya terdapat 20 identifier pada pfa00010. `INNER JOIN` mempertahankan 19 identifier yang mempunyai pasangan pada `go_annotation`, lalu setiap rekaman anotasi menghasilkan baris tersendiri.

Mengapa DISTINCT bukan obat untuk semua pengulangan. Pengulangan identifier dapat benar secara relasional. Jika satu identifier mempunyai m anotasi dan n hubungan *pathway*, penggabungan dua tabel anak melalui `gene_id` dapat menghasilkan $m \times n$ kombinasi untuk identifier tersebut. Tentukan terlebih dahulu apakah satuan yang diperlukan adalah identifier, rekaman anotasi, atau pasangan anotasi-*pathway*.

Jika yang dicari hanya daftar identifier, pilih `DISTINCT` pada kolom identifier. Jika yang dicari adalah rekaman anotasi beserta buktinya, menghapus baris yang tampak mirip dapat menghilangkan informasi. Rekaman GO yang sama dapat mempunyai rujukan atau bukti yang berbeda.

Mendiagnosis hasil dengan LEFT JOIN. Ubah `INNER JOIN` pada query utama menjadi `LEFT JOIN` sambil mempertahankan filter pfa00010. Perhatikan identifier tambahan dengan kolom anotasi bernilai NULL. Selanjutnya, bandingkan `COUNT(*)` dan `COUNT(a.annotation_id)` pada versi `LEFT JOIN`. Gunakan perbedaan kedua hitungan tersebut untuk menjelaskan baris yang mempunyai pasangan anotasi dan baris yang hanya dipertahankan oleh `LEFT JOIN`. Contoh pertambahan baris pada `JOIN` anotasi ditunjukkan pada Gambar 13.

	gene_id text	pathway_id text	go_id text	evidence_code text	gene_product_name text
1	PF3D7_0318800	pfa00010	GO:0004807	ISS	triosephosphate isomerase, putative
2	PF3D7_0318800	pfa00010	GO:0006096	ISS	triosephosphate isomerase, putative
3	PF3D7_0624000	pfa00010	GO:0004396	IDA	hexokinase
4	PF3D7_0624000	pfa00010	GO:0005634	HDA	hexokinase
5	PF3D7_0624000	pfa00010	GO:0005829	IDA	hexokinase
6	PF3D7_0624000	pfa00010	GO:0006096	IDA	hexokinase
7	PF3D7_0624000	pfa00010	GO:0009410	IMP	hexokinase
8	PF3D7_0624000	pfa00010	GO:0020036	RCA	hexokinase
9	PF3D7_0624000	pfa00010	GO:0036396	HDA	hexokinase
10	PF3D7_0624000	pfa00010	GO:0003722	IDA	hexokinase

Total rows: 103 Query complete 00:00:00.120

Gambar 13. Banyak baris untuk satu identifier dapat mencerminkan banyak rekaman anotasi dan bukan kesalahan database.

Tugas: Diagnosis Hasil JOIN Anotasi

Jelaskan mengapa pernyataan “*pathway* ini memiliki 103 gen” tidak sesuai dengan hasil query. Gunakan jumlah baris, jumlah identifier berbeda, dan hubungan satu-ke-banyak antara identifier dan anotasi untuk mendukung penjelasan.

16 Mengatasi Kendala yang Sering Muncul

Gunakan Tabel 7 untuk memeriksa kendala umum sebelum mengulang proses atau mengubah data. Pada tahap pemulihan, hindari tindakan yang menghapus objek secara paksa tanpa pendampingan asisten.

Tabel 7. Kendala umum dan pemeriksaan yang disarankan.

Gejala	Pemeriksaan dan tindakan
<code>schema public already exists</code> saat <i>restore</i>	Pastikan database tujuan baru dan kosong. Jalankan penghapusan <code>public</code> dengan <code>RESTRICT</code> pada database tujuan, lalu ulangi <i>restore</i> . Jangan lakukan pada database induk.
<code>cannot drop schema</code> karena berisi objek	Hentikan proses. Minta asisten memeriksa isi database atau membuat database tujuan baru. Jangan memakai <code>CASCADE</code> .
<i>Restore</i> sukses tetapi tabel kosong	Periksa apakah <i>Data</i> aktif bersama <i>Pre-data</i> dan <i>Post-data</i> . Jika perlu, ulangi pemulihan pada database baru yang kosong dengan pendampingan asisten.
<code>relation already exists</code> atau <code>duplicate key</code> saat <i>restore</i>	Kemungkinan <i>restore</i> diulang pada database yang sudah terisi. Jangan menghapus baris untuk mengakali galat; gunakan database tujuan kosong.
<code>relation does not exist</code> saat <code>SELECT</code>	Periksa <code>current_database()</code> , keberadaan <code>public</code> , ejaan tabel, dan keberhasilan <i>restore</i> . <i>Refresh</i> hanya memperbarui tampilan; tidak menciptakan tabel.
<code>column reference is ambiguous</code>	Awali kolom dengan alias tabel, misalnya <code>r.gene_id</code> atau <code>gp.gene_id</code> .
<code>syntax error</code>	Baca posisi galat pada panel <i>Messages</i> . Periksa koma, tanda petik tunggal, tanda kurung, dan bagian query yang disorot.
Hasil hanya 20 baris	Periksa <code>LIMIT</code> dan batas baris pada toolbar. Gunakan <code>COUNT</code> untuk mengetahui jumlah penuh.
Hasil lebih banyak dari perkiraan	Periksa kardinalitas dan kondisi <code>ON</code> . Satu identifier dapat mempunyai banyak anotasi atau <i>pathway</i> .
Hasil kosong dengan <code>= NULL</code>	Gunakan <code>IS NULL</code> . Jangan menyamakan <code>NULL</code> dengan 0 atau teks kosong.
<code>pg_restore</code> tidak ditemukan atau versi tidak cocok	Minta asisten memeriksa <i>Preferences</i> → <i>Paths</i> → <i>Binary paths</i> dan utilitas PostgreSQL yang terpasang. Gunakan paket versi laboratorium yang telah diuji.

Sebelum meminta bantuan, catat nama database aktif, query lengkap atau nama proses yang dijalankan, pesan galat utuh, dan langkah terakhir yang berhasil. Sertakan tangkapan layar yang memperlihatkan konteks tersebut dan jangan membagikan kata sandi. Setelah perbaikan *restore*, ulangi pemeriksaan jumlah baris dan *constraint* sebelum melanjutkan.

17 Latihan Individual dan Refleksi

Kerjakan latihan berikut setelah seluruh titik pemeriksaan dasar sesuai. Untuk setiap latihan, tuliskan pertanyaan, prediksi, SQL atau rancangan, hasil beserta satuannya, dan interpretasi. Diskusi penalaran diperbolehkan, tetapi setiap mahasiswa menyimpan skrip dan menulis penjelasannya sendiri.

17.1 Latihan Individual

Latihan A: Memilih arah penggabungan. Tampilkan seluruh identifier RNA-seq beserta skor *fitness* mutagenesis jika tersedia. Temukan identifier RNA-seq yang belum mempunyai pasangan skor. Pilih jenis JOIN, jelaskan sisi yang dipertahankan, dan hitung identifier yang tidak cocok. Nilai skor yang tidak tersedia tidak boleh diartikan sebagai nilai nol.

Latihan B: Menguji FULL OUTER JOIN. Gunakan FULL OUTER JOIN antara *reads* dan *mis_essential*. Bandingkan jumlah barisnya dengan LEFT JOIN yang mempertahankan *reads*. Jelaskan mengapa hasilnya dapat sama meskipun FULL OUTER JOIN mempertahankan kedua sisi. Gunakan pemeriksaan data untuk mendukung jawaban.

Latihan C: Menggabungkan dan merangkum. Tampilkan seluruh *pathway* dan jumlah identifier pada setiap *pathway* yang mempunyai data RNA-seq. Pertahankan *pathway* meskipun jumlah identifier yang cocok nol. Gunakan dua LEFT JOIN, hitung kolom kunci dari *reads*, lalu urutkan hasil dari jumlah tertinggi. Jelaskan perbedaan makna antara menghitung *gp.gene_id* dan *r.gene_id*.

Latihan D: Memindahkan pola berpikir. Rancang sketsa tiga tabel untuk kasus biologis lain, misalnya senyawa–hasil uji–target protein. Tentukan arti satu baris pada setiap tabel, *primary key*, *foreign key*, dan kardinalitas hubungannya. Tuliskan satu pertanyaan yang memerlukan INNER JOIN dan satu pertanyaan yang memerlukan LEFT JOIN. Latihan ini tidak memerlukan pembuatan database baru.

17.2 Refleksi Akhir

Jawab dalam satu atau dua paragraf: apa yang berubah pada hasil ketika jenis JOIN, kondisi ON, atau lokasi WHERE diubah? Jelaskan bagaimana pola penalaran tersebut dapat membantu ketika merancang atau menganalisis basis data selain BioSQLite.

18 Lembar Kerja Praktikum

Lembar Kerja Praktikum (LKP) memuat tujuan singkat, model data, bukti *restore*, hasil utama percobaan terpandu, Latihan A–D, refleksi, dan referensi. Seluruh query yang berhasil disimpan dalam berkas SQL.

18.1 Berkas yang Dikumpulkan dan Skrip SQL

Setiap mahasiswa mengumpulkan dua berkas pada Tabel 8. Penamaan mengikuti ketentuan khusus pada draf Praktikum 05.

Tabel 8. Berkas yang dikumpulkan pada Praktikum 05.

Berkas	Isi utama
NIM_Nama_Praktikum05.sql	Perintah SQL yang berhasil, disusun berurutan, dengan komentar penanda bagian.
NIM_Nama_Laporan05.pdf	Dokumentasi hasil, pembahasan, Latihan A–D, dan refleksi praktikum.

Isi skrip SQL.

Susun NIM_Nama_Praktikum05.sql sesuai urutan kegiatan dan gunakan komentar untuk menandai setiap bagian. Skrip memuat pemeriksaan database aktif, validasi hasil *restore*, *query filtering* dan *sorting*, berbagai jenis JOIN, penempatan filter pada ON dan WHERE, agregasi, GROUP BY, HAVING, *subquery*, penggabungan anotasi GO, serta jawaban SQL Latihan A–C. Latihan D berupa rancangan model data dan dicantumkan dalam laporan.

Gunakan komentar dengan awalan `--` untuk memberi label bagian dan menjelaskan tujuan singkat *query*. Hasil, interpretasi, dan batas kesimpulan utama ditulis dalam laporan. Contoh penulisan komentar ditunjukkan berikut.

```
-- Bagian 7: INNER JOIN RNA-seq dan mutagenesis
-- Tujuan: menampilkan identifier yang tersedia pada reads dan mis_essential
```

18.2 Struktur dan Ketentuan Laporan Praktikum

Laporan menjadi dokumentasi penerapan SQL lanjut untuk menjawab pertanyaan berdasarkan data BioSQLite. Struktur dan isi minimal laporan dirangkum pada Tabel 9.

Tabel 9. Struktur laporan Praktikum SQL Lanjut dan Query Biologis.

No.	Komponen	Isi minimal
1	Cover	Judul laporan, identitas mata kuliah, nama dan NIM mahasiswa, program studi, fakultas, universitas, serta tahun akademik.
2	Daftar Isi	Seluruh judul dan subjudul laporan beserta nomor halaman.
3	Model Data dan Persiapan Database	Ringkasan tujuh tabel, ERD, relasi kunci, serta bukti <i>restore</i> dan validasi database.
4	Hasil Utama Percobaan Terpandu dan Tugas Modul	Jawaban seluruh kotak hitam berlabel Tugas pada Bagian 7–15. Jawaban tersebut mencakup hasil terpilih yang menunjukkan nilai NULL atau baris tanpa pasangan, cakupan FULL OUTER JOIN, dan penambahan baris pada JOIN anotasi, disertai SQL, jumlah baris beserta satuannya, dan interpretasi.
5	Latihan Individual A–D	Pertanyaan, prediksi, SQL atau rancangan, hasil, dan interpretasi setiap latihan individual.
6	Refleksi	Perubahan hasil akibat pemilihan JOIN, kondisi ON, dan WHERE, serta penerapan pola penalaran pada kasus lain.
7	Daftar Pustaka	Sumber yang benar-benar dirujuk, apabila digunakan.

Gunakan cuplikan hasil yang relevan dan hindari menempelkan seluruh tabel berukuran besar.

Ketentuan teknis.

Penulisan laporan mengikuti PPKI. Ketentuan teknis laporan dirangkum pada Tabel 10.

Tabel 10. Ketentuan teknis laporan Praktikum 05.

Komponen	Ketentuan
Format berkas	Laporan dikumpulkan dalam format PDF dan skrip dalam format .sql.
Margin halaman	Kiri 4 cm; atas, kanan, dan bawah masing-masing 3 cm.
Teks isi	Ukuran huruf 12 pt dengan spasi 1,5.
Judul bagian	Ukuran huruf judul bagian 14 pt.
Daftar isi	Memuat judul, subjudul, dan nomor halaman.
Gambar dan tangkapan layar	Memiliki nomor dan keterangan di bawah gambar serta terbaca pada ukuran halaman.
Tabel	Memiliki nomor dan keterangan di atas tabel; tabel lintas halaman menggunakan kepala tabel berulang dan judul “(lanjutan)”.